Tree-based Intelligent Intrusion Detection System in Internet of Vehicles

Paper Reproduction & Critical Review

Riham Badarna
Based on the paper by Li Yang, Abdallah Moubayed, Ismail Hamieh and Abdallah Shami

June 25, 2026

1 Executive Summary

This project reproduces and critically evaluates the paper *Tree-based Intelligent Intrusion Detection System in Internet of Vehicles*. The objective of the original work is to improve intrusion detection by combining ensemble feature selection with a stacking ensemble of tree-based machine learning classifiers. The reproduction focuses on verifying the methodology, evaluating its reproducibility, and assessing whether the reported conclusions are supported by experimental evidence.

The experiments were conducted using the CICIDS2017 sample dataset provided with the authors' implementation. The complete machine learning pipeline was reproduced, including data exploration, preprocessing, feature engineering, ensemble feature selection, model training, error analysis, and performance evaluation. To strengthen the evaluation, an additional validation experiment was conducted after removing duplicated observations from the sampled dataset to assess their influence on the reported performance. Five classifiers were implemented: Decision Tree, Random Forest, Extra Trees, XGBoost, and a stacking ensemble. Model performance was evaluated using Accuracy, Precision, Recall, F1-score, Matthews Correlation Coefficient (MCC), execution time, and confusion matrix analysis.

The reproduced results followed the main performance trends reported in the original paper. Similar to the published work, XGBoost achieved the highest performance among the individual classifiers, while the stacking ensemble obtained the best overall results. Although the reproduced performance metrics were slightly lower than those reported by the authors, the ranking of the classifiers remained consistent, confirming the effectiveness of the proposed ensemble learning framework. The duplicate-removal validation experiment further showed that removing duplicated observations caused only a small decrease in Accuracy, F1-score, and MCC, indicating that duplicated flows slightly inflated the reported performance but did not alter the overall conclusions.

A critical evaluation of the paper identified several strengths and limitations. The proposed methodology achieves excellent intrusion detection performance while reducing the feature space through ensemble feature selection. However, several methodological limitations were observed, including the exclusive use of tree-based base learners in the stacking ensemble, the lack of justification for the selected 90% cumulative feature importance threshold, the absence of duplicate-handling procedures despite the presence of duplicated observations in the sampled dataset, and the absence of validation on independent or temporally evolving datasets.

Overall, this reproduction supports the authors' main conclusions while highlighting opportunities for improving the robustness, transparency, and generalizability of the proposed intrusion detection framework. The study demonstrates that the proposed methodology provides a strong baseline for supervised intrusion detection and offers several directions for future research.

2 Summary of the Source

2.1 Problem Being Addressed

The paper addresses the problem of intrusion detection in the Internet of Vehicles (IoV). Modern vehicles are becoming increasingly connected through Vehicle-to-Everything (V2X) technologies, which allow communication between vehicles, infrastructure, pedestrians, and cloud services. While this connectivity improves transportation efficiency and supports autonomous driving, it also exposes vehicles to various cyber-attacks. These attacks can target both external communication networks and the internal Controller Area Network (CAN) bus used by the vehicle's electronic control units (ECUs). Examples of such attacks include Denial-of-Service (DoS), spoofing, port scanning, brute-force attacks, web attacks, and CAN message injection attacks.

2.2 Why the Problem is Important

Cybersecurity is a critical challenge in autonomous and connected vehicles because successful attacks can have consequences that go beyond data loss or financial damage. A compromised vehicle may malfunction, receive false information, or even be maliciously controlled, potentially putting passengers and other road users at risk. Since vehicles continuously exchange information with many devices and services, traditional security mechanisms alone may not be sufficient. Therefore, effective intrusion detection systems are needed to automatically identify malicious activities and protect both intra-vehicle and external communications.

2.3 Proposed Solution

The authors propose an intelligent Intrusion Detection System (IDS) based on tree-structured machine learning algorithms. The system is designed to detect a wide range of attacks in both vehicle internal networks (CAN bus) and external communication networks. To improve detection performance, the authors combine multiple machine learning models using a stacking ensemble approach. In addition, they introduce a feature selection method based on averaging feature importance scores from multiple tree-based models. This reduces computational cost while maintaining high detection accuracy.

2.4 Dataset Used

The study evaluates the proposed IDS using two publicly available datasets:

1. Car-Hacking (CAN Intrusion) Dataset – This dataset contains CAN bus traffic collected from vehicles and includes normal traffic as well as attacks such as DoS, fuzzy attacks, RPM spoofing, and gear spoofing.
2. CICIDS2017 Dataset – A widely used intrusion detection dataset that contains network traffic representing both normal activity and several cyber-attacks, including DoS, port scanning, brute-force attacks, web attacks, botnet attacks, and infiltration attacks.

Using both datasets allows the authors to evaluate the proposed IDS on both internal vehicle communications and general network traffic.

The original paper evaluates the proposed framework on the complete CICIDS2017 dataset. However, the authors also provide a sampled version of the dataset together with their implementation to facilitate reproduction.

2.5 Model and Methodology

The proposed methodology consists of several stages. First, the network traffic data is collected and preprocessed. Numerical features are normalized, and class imbalance is addressed using the Synthetic Minority Oversampling Technique (SMOTE). Next, feature selection is performed using an averaging feature importance approach derived from multiple tree-based models. The classification stage employs four machine learning algorithms:

- Decision Tree (DT)

- Random Forest (RF)
- Extra Trees (ET)
- Extreme Gradient Boosting (XGBoost)

These models are first trained individually and then combined using a stacking ensemble method to improve overall performance. The system is evaluated using 5-fold cross-validation and performance metrics including Accuracy, Detection Rate (Recall), False Alarm Rate, F1-Score, and Execution Time. The results demonstrate that the proposed IDS achieves very high intrusion detection accuracy while maintaining reasonable computational efficiency.

3 Critical Evaluation

3.1 Main Claims Made by the Authors

The authors claim that their proposed tree-based Intrusion Detection System (IDS) can effectively detect cyber-attacks in both internal vehicle networks and external communication networks. They argue that combining multiple tree-based classifiers through a stacking ensemble improves detection performance compared to individual classifiers. In addition, they claim that their ensemble feature-selection method reduces computational cost while maintaining high classification accuracy, making the framework suitable for real-time intrusion detection.

3.2 Are the Claims Supported by the Presented Evidence?

The authors evaluate the proposed framework on two widely used benchmark datasets: the Car-Hacking (CAN Intrusion) Dataset and CICIDS2017. The reported results show very high classification performance, with accuracies close to 100% for both the individual tree-based models and the stacking ensemble. The feature-selection method also reduces the execution time while causing only a small decrease in predictive performance.

The reproduction performed in this study supports the main trends reported by the authors. Similar to the original work, the stacking ensemble achieved the best overall performance and XGBoost was the strongest individual classifier. Furthermore, the additional duplicate-removal validation experiment showed that removing 22.2% duplicated observations caused only a small decrease in Accuracy, F1-score, and MCC, indicating that the reported performance is not solely explained by duplicated network flows.

Nevertheless, the evidence should be interpreted with some caution. The reproduction was conducted using the sampled CICIDS2017 dataset released with the authors' implementation rather than the complete dataset used in the paper, making direct numerical comparison inappropriate. In addition, the Car-Hacking dataset experiments could not be reproduced because the corresponding implementation was not provided, and the original paper does not discuss whether duplicated observations were removed before evaluation. Finally, the experiments are limited to benchmark datasets and therefore provide limited evidence regarding the framework's performance in real-world Internet of Vehicles environments or under evolving attack patterns.

3.3 Is the Evaluation Methodology Appropriate?

The evaluation methodology is generally appropriate for the intrusion detection problem. The authors evaluate the framework on two datasets representing both internal vehicle communication attacks and external network attacks. They compare the proposed approach against several baseline machine learning methods using multiple evaluation metrics, including Accuracy, Detection Rate, False Alarm Rate, F1-score, and Execution Time.

Nevertheless, several methodological aspects could have been described more clearly. The paper states that SMOTE was applied to address class imbalance but does not explicitly describe whether oversampling was performed only after the training-test split. Since applying SMOTE before splitting the data would introduce data leakage, providing this implementation detail would improve both reproducibility and methodological transparency.

3.4 Possible Weaknesses or Limitations

Despite the strong reported results, several limitations should be considered. First, all four base learners used in the stacking ensemble belong to the same family of tree-based classifiers. Although these algorithms differ in their implementation, they share similar inductive biases, potentially limiting the diversity that stacking ensembles typically benefit from.

Second, the feature selection procedure uses a cumulative feature importance threshold of 90%, but the paper does not justify why this particular threshold was selected or investigate whether alternative thresholds could provide a better trade-off between predictive performance and computational efficiency.

Third, the evaluation relies exclusively on two benchmark datasets collected under controlled conditions. During our exploratory analysis of the CICIDS2017 sample dataset, approximately 22% of the observations were found to be duplicated, together with several constant and highly correlated features. The paper does not discuss how such dataset characteristics might influence the reported performance or affect the generalizability of the proposed framework.

Fourth, the evaluation focuses on static benchmark datasets and does not investigate concept drift or evolving attack patterns. Since cyber-attacks in Internet of Vehicles environments continuously evolve over time, additional experiments on more recent or continuously collected datasets would strengthen the evidence for the proposed framework.

Finally, although the paper reports its experimental results on the complete CICIDS2017 dataset, the accompanying implementation also provides a sampled dataset for reproduction. A clearer distinction between the datasets used for the published experiments and those used in the released implementation would improve reproducibility and reduce ambiguity for future researchers.

3.5 Are the Conclusions Justified?

Based on the evidence presented in the paper, the authors' conclusions are generally justified. The experimental results consistently show that the stacking ensemble outperforms the individual classifiers, while the ensemble feature selection strategy reduces the number of input features without substantially affecting classification performance. However, the conclusions should be interpreted within the scope of the evaluated datasets. Additional validation on independent datasets, together with experiments addressing temporal generalization and alternative feature-selection thresholds, would provide stronger evidence that the proposed framework generalizes beyond the benchmark datasets.

4 Feature Engineering Analysis

Feature engineering plays an important role in the proposed IDS framework. Although the authors do not create entirely new domain-specific features, they perform several preprocessing and feature-selection steps that directly affect model performance. Their approach focuses on improving data quality, handling class imbalance, reducing redundancy, and identifying the most informative traffic characteristics for intrusion detection. Therefore, feature engineering is a significant component of the proposed methodology rather than a secondary preprocessing step.

The datasets used in the study contain numerous network traffic features extracted from vehicle communication and network flows. Examples of features include Flow Duration, Destination Port, Average Packet Size, Packet Length statistics, TCP flag counts, Subflow Bytes, and various packet-level measurements. The CICIDS2017 dataset originally contains 78 traffic features, while the Car-Hacking dataset contains CAN bus communication features representing both normal vehicle operation and different attack types. These features capture network behavior, communication patterns, packet characteristics, and protocol information that can help distinguish normal traffic from malicious activity.

Several feature engineering and preprocessing techniques were applied before training the machine learning models. First, the authors applied Min-Max normalization to scale numerical features into a common range. This transformation was necessary because different network features have very different numerical scales. For example, packet counts, flow durations, and byte statistics may vary by several orders of magnitude. Normalization prevents features with large numeric values from dominating the learning process and improves the consistency of the input data. The authors also addressed class imbalance using the Synthetic Minority Oversampling Technique (SMOTE). Although

SMOTE is not a feature transformation in the traditional sense, it is an important preprocessing step because some attack categories contain far fewer samples than normal traffic. By generating synthetic minority-class observations, SMOTE helps the classifiers learn attack patterns more effectively and reduces bias toward the majority class.

The most important feature engineering contribution of the paper is the feature-selection procedure. The authors trained four tree-based models—Decision Tree, Random Forest, Extra Trees, and XGBoost—and extracted feature-importance scores from each model. Instead of relying on a single model's ranking, they averaged the importance values across all four classifiers. Features were then ranked according to their average importance, and only those contributing to 90% of the cumulative importance were retained. This process reduced the number of features significantly while preserving nearly the same predictive performance. Experimental results show that the reduction in features led to substantial decreases in execution time with only minimal reductions in classification accuracy, demonstrating that the selected features captured most of the useful information in the datasets.

Redundancy likely exists in the original datasets. Network traffic datasets often contain groups of highly related variables, such as packet length mean, packet length variance and average packet size. These features may capture similar information and therefore introduce redundancy. Redundancy can be identified using correlation analysis, variance inflation factor (VIF) analysis, mutual information, or feature-importance rankings. The authors address this issue through feature selection, which removes low-importance variables. Additional analysis could include correlation heatmaps to identify highly correlated features and eliminate unnecessary duplication while preserving predictive power.

As will be discussed in the exploratory data analysis, redundancy indeed exists within the dataset. Approximately 22% of the observations were duplicated, eight features contained a single constant value, and many numerical features exhibited strong correlations.

The feature engineering process is meaningful from both a machine learning and cybersecurity perspective. From a machine learning viewpoint, normalization improves data consistency, SMOTE addresses class imbalance, and feature selection reduces dimensionality and computational cost. From a cybersecurity perspective, the selected features correspond to characteristics that naturally change during attacks. For example, packet lengths, packet size statistics, destination ports, and flow characteristics are directly related to how attackers interact with networks. The fact that these features consistently appear among the most important predictors suggests that the model is learning meaningful attack behavior rather than relying on arbitrary patterns in the data.

Several additional features could potentially improve performance in future work. Temporal features such as connection frequency within short time windows, rolling packet-rate statistics, and session-based behavioral metrics could provide additional information about attack progression. Features derived from communication history, protocol transitions, or anomaly scores generated by unsupervised methods may also help detect previously unseen attacks. Furthermore, graph-based features describing communication relationships between devices could be valuable in Internet of Vehicles environments, where interactions among vehicles, infrastructure, and external services form complex network structures.

5 Reproducibility Analysis

5.1 Code Execution

The authors provide Jupyter notebook implementations for the proposed intrusion detection framework. The repository contains notebooks corresponding to multiple papers from the same research group, including the Tree-Based IDS, MTH-IDS, and LCCDE approaches. The code is written in Python and relies on widely used machine learning libraries such as Scikit-learn, XGBoost, NumPy, and Pandas. Based on the provided implementation, the code can be executed successfully once the required datasets and dependencies are available.

5.2 Availability of Required Files and Dependencies

Most of the required files are available in the repository. The repository includes sampled versions of the CICIDS2017 dataset as well as the notebooks used for training and evaluation. The Python

packages used by the implementation are common machine learning libraries that can be installed easily through standard package managers.

However, an important observation is that the paper reports experiments on two datasets: the Car-Hacking (CAN Intrusion) Dataset and CICIDS2017. While the paper discusses results for both datasets, the publicly available notebooks only contain code that loads and processes CICIDS2017-derived datasets. No implementation corresponding to the Car-Hacking Dataset experiments was found in the released notebooks. Therefore, the repository provides only a partial reproduction of the experiments described in the paper.

5.3 Hidden Preprocessing Steps

The paper describes several preprocessing operations, including data cleaning, relabeling, Min-Max normalization, SMOTE oversampling, and feature selection based on feature importance. Most of these steps are implemented explicitly in the notebooks and can be followed by examining the code.

Nevertheless, there are indications that some preprocessing may have been performed before the released datasets were generated. For example, the notebooks use sampled and preprocessed versions of CICIDS2017 rather than the original raw dataset. Additionally, the paper states that the datasets were modified through data combination, missing-value removal, and relabeling before experimentation. Because the repository does not provide the complete pipeline for generating these processed datasets from the original sources, some preprocessing decisions remain difficult to verify independently.

5.4 Overall Reproducibility

Overall, the paper demonstrates good but not perfect reproducibility. The authors provide code, datasets, and detailed methodological descriptions, making it possible to reproduce the main CICIDS2017 experiments. The machine learning models, preprocessing procedures, feature-selection strategy, and evaluation metrics are described clearly enough for independent implementation.

However, reproducibility is weakened by the absence of publicly available code for the Car-Hacking Dataset experiments reported in the paper. As a result, only part of the published experimental evaluation can be reproduced directly from the repository. Despite this limitation, the work remains substantially more reproducible than many machine learning studies because the core implementation, datasets, and experimental procedures are publicly accessible.

6 Experimental Reproduction

6.1 Data Loading

The dataset used in this study is the CICIDS2017 sample dataset, which contains 56,661 network traffic flows described by 78 columns. Among these columns, 77 are numerical features and one column represents the target label. The dataset includes seven classes: BENIGN, DoS, PortScan, BruteForce, WebAttack, Bot, and Infiltration. The class distribution is highly imbalanced, with only 36 Infiltration samples compared to 22,731 BENIGN samples. This observation is consistent with real-world cybersecurity scenarios, where malicious traffic is typically much rarer than normal network activity.

Unlike the original paper, which reports its experimental results on the complete CICIDS2017 dataset, this reproduction uses the sampled dataset released with the authors' implementation. Consequently, the experiments presented in this report should be interpreted as a sample-based reproduction of the proposed methodology rather than a direct numerical replication of the full-dataset results reported in the paper.

To evaluate whether duplicated observations affected the validity of the reported performance, an additional experiment was conducted after removing all duplicate rows. The complete preprocessing, feature engineering, and model training pipeline was then repeated on the duplicate-removed dataset, allowing the impact of duplicate observations on classification performance to be assessed.

Inspection of the feature names indicates that the dataset contains network-flow statistics commonly used in intrusion detection systems. Examples include packet counts, packet lengths, flow duration, packet rates, TCP flag statistics, and flow-level communication metrics. These features

are meaningful from a cybersecurity perspective because many attack types generate traffic patterns that differ from normal network behavior. The column names are descriptive and consistent with the original CICIDS2017 dataset.

The dataset uses a standard RangeIndex that uniquely identifies each observation. The index values are sequential and do not contain additional information relevant to the prediction task. Therefore, the index should not be treated as a predictive feature and can safely be ignored during modeling.

Temporal analysis revealed that the dataset contains only one explicit temporal feature, namely "Flow Duration". No timestamps or date-time variables are available. Consequently, it is not possible to analyze temporal trends over time, but flow duration itself may still provide useful information for distinguishing between normal and malicious traffic.

Missing-value analysis showed that the dataset is largely complete. Only 54 missing values were found initially, all within the "Flow Bytes/s" feature. In addition, 108 infinite values were detected and converted to missing values, resulting in a total of 162 missing entries. The small number of missing values suggests that substantial preprocessing and cleaning were performed before the dataset was released.

Data quality checks revealed 12,598 duplicated rows, representing approximately 22% of the dataset. At this stage, duplicates were retained because it is unclear whether they correspond to repeated legitimate network flows or are artifacts of the sampling process. Further investigation will determine whether removing duplicates affects model performance.

Eight features were found to contain a single constant value across all observations: Bwd PSH Flags, Bwd URG Flags, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, and Bwd Avg Bulk Rate. Such features do not provide discriminative information and are strong candidates for removal during feature engineering.

Finally, no duplicated feature names were detected. However, several features appear conceptually related, such as packet-length statistics and packet-size metrics. Correlation analysis in the EDA stage will be used to determine whether some of these variables provide redundant information and should be removed or combined.

6.2 Exploratory Data Analysis

6.2.1 Feature Distribution

Feature distribution analysis was performed for all non-constant numerical features to better understand the characteristics of the CICIDS2017 sample dataset. As shown in Figure 1, most features exhibit highly right-skewed distributions, where the majority of observations are concentrated near zero and only a small number of observations take very large values. Several features also show multiple peaks, indicating different traffic patterns within the dataset. This behavior is expected because the dataset contains both benign traffic and multiple attack types, each with different network characteristics. The observed skewness suggests that normality assumptions do not hold for many features, supporting the use of Spearman correlation during EDA and tree-based models, as used by the authors, which are generally robust to skewed data distributions.

6.2.2 Missing Values

Missing value analysis showed that the dataset is highly complete. After replacing infinite values with missing values, only two features (Flow Bytes/s and Flow Packets/s) contained missing values, with 81 missing entries each. These missing values most likely resulted from divisions by zero during the computation of rate-based features, such as bytes or packets per second.

Overall, the proportion of missing values is extremely small compared to the dataset size (56,661 samples), indicating that the released dataset has already undergone substantial preprocessing and cleaning. Therefore, missing values are not expected to significantly affect model performance and can be handled during the preprocessing stage.

6.2.3 Outlier Analysis

Outlier analysis was performed using the Interquartile Range (IQR) method on all non-constant numerical features. As shown in Figure 5.3, the highest outlier percentages were observed in Idle Max,

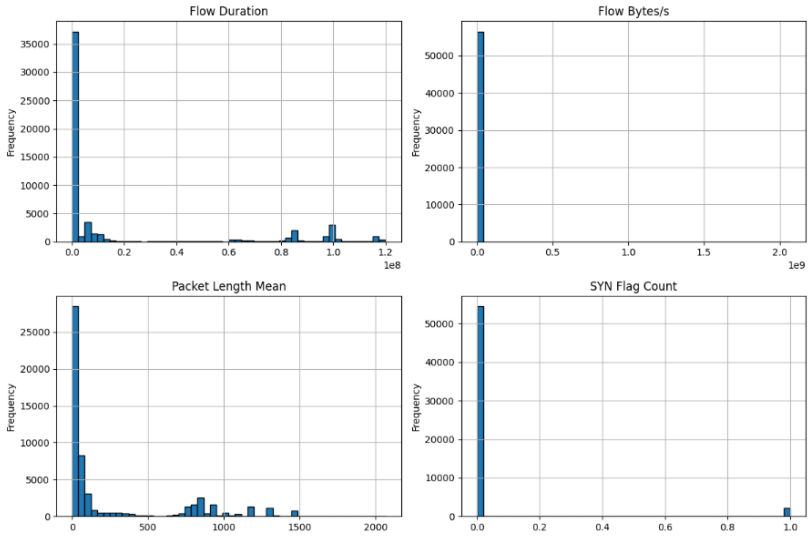

Figure 1: Representative feature distributions. The selected features illustrate the highly skewed nature of network traffic variables, the presence of long-tailed distributions, and the differences between continuous traffic measurements and discrete protocol-based features.

Idle Min, Idle Mean, and the Active timing features, each containing approximately 24–25% outliers. Packet-related features, such as Packet Length Variance and Max Packet Length, also exhibited relatively high outlier rates.

These results are expected for a network intrusion dataset. Network attacks often generate unusually large packet sizes, long idle periods, or abnormal communication patterns, causing some observations to naturally fall far from the majority of the data. Therefore, these outliers should not automatically be considered errors or removed, as they may represent the malicious behavior that the intrusion detection system is designed to identify. This observation also supports the authors' choice of tree-based machine learning models, which are generally robust to outliers and do not require strict assumptions about the data distribution.

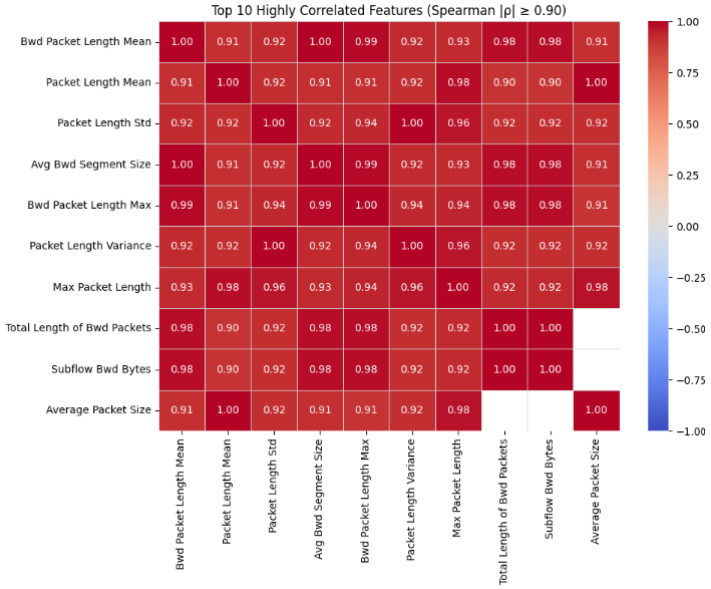

Figure 2: Spearman correlation heatmap for the top 10 highly correlated features $|\rho| \geq 0.90$. The figure illustrates the strong positive relationships among packet length, packet size, and byte-related features, indicating substantial feature redundancy within the dataset.

6.2.4 Correlation Analysis

Correlation analysis was performed to identify relationships and potential redundancy among the network traffic features. Although the original paper does not use correlation for feature selection, instead relying on tree-based feature importance, this analysis provides additional insight into the structure of the dataset.

Both Pearson and Spearman correlation coefficients were computed. Pearson correlation measures the strength of linear relationships between continuous variables and is more sensitive to outliers and non-normal data. Spearman correlation measures monotonic relationships using feature ranks, making it more robust to skewed distributions and extreme values. Since the feature distribution analysis revealed highly skewed variables with numerous outliers, Spearman correlation was selected as the primary measure for interpretation. Kendall correlation was not used because it is computationally more expensive and is generally preferred for smaller datasets, while Spearman provides similar rank-based information for large datasets such as CICIDS2017.

The results revealed substantial redundancy among the network traffic features. Using a threshold of $|\rho| \geq 0.50$, Spearman identified 953 highly correlated feature pairs compared to 276 using Pearson, indicating that many relationships are monotonic rather than strictly linear. At the stronger threshold of $|\rho| \geq 0.90$, the two methods produced similar results (114 pairs for Spearman and 102 pairs for Pearson), showing that many feature relationships remain highly correlated regardless of the correlation measure. As illustrated in Figure 2, the selected highly correlated features form distinct clusters of strong positive correlations, particularly among packet length, packet size, and byte-related features. Several feature pairs exhibited correlations close to 1.0, including Total Fwd Packets – Subflow Fwd Packets, Fwd Packet Length Mean – Avg Fwd Segment Size, and Packet Length Mean – Average Packet Size, suggesting that these variables capture nearly identical information.

From a cybersecurity perspective, these findings are expected because many CICIDS2017 features are derived from the same packet, byte, and timing statistics. Consequently, strong correlations reflect genuine relationships within network traffic rather than data quality issues. The strong correlation patterns shown in Figure 2 support the authors' decision to apply feature selection before model training, reducing feature redundancy while preserving most of the predictive information.

6.2.5 Temporal Features

The dataset contains only one explicit temporal feature, Flow Duration, representing the duration of each network flow. As shown in Figure 5.4, the distribution is highly right-skewed, with most flows having relatively short durations and a small number of flows lasting much longer. This observation is also reflected in the summary statistics, where the median flow duration (58,897) is considerably lower than the mean (19.4 million), indicating the presence of long-duration flows.

This behavior is consistent with real-world network traffic, where most connections are brief while only a small fraction correspond to long communication sessions or potential attacks. Although the dataset does not include timestamps that would allow analyzing attacks over time, Flow Duration still provides valuable temporal information for distinguishing between benign and malicious traffic.

6.2.6 Class Imbalance

The class distribution analysis revealed a highly imbalanced dataset. As shown in Table 1, the BENIGN class accounts for 40.12% of the samples, while the Infiltration class represents only 0.06%, resulting in an imbalance ratio of approximately 631:1. Such imbalance is expected in real-world cybersecurity, where legitimate network traffic is far more common than malicious activity, particularly for rare attack types.

A highly imbalanced dataset may bias machine learning models toward the majority classes, making minority attacks more difficult to detect. Therefore, relying solely on overall accuracy could lead to misleading performance estimates. The authors addressed this issue by applying SMOTE (Synthetic Minority Oversampling Technique) during preprocessing to generate synthetic samples for minority classes before training the classifiers. Based on the observed class distribution, this preprocessing step is well justified and likely contributes to improving the detection of rare attacks.

Table 1: Class distribution of the CICIDS2017 sample dataset.

Label	Count	Percentage (%)
BENIGN	22,731	40.12
DoS	19,035	33.59
PortScan	7,946	14.02
BruteForce	2,767	4.88
WebAttack	2,180	3.85
Bot	1,966	3.47
Infiltration	36	0.06

6.3 Feature Engineering

6.3.1 Encoding of Categorical Variables

The target variable, representing the network traffic class (e.g., BENIGN, DoS, Bot, and WebAttack), was encoded into numerical labels using LabelEncoder, allowing it to be used by the machine learning algorithms. No additional encoding was required for the predictor variables since all network traffic features in the CICIDS2017 dataset are already numerical. This preprocessing step preserves the original information while ensuring compatibility with the learning algorithms.

6.3.2 Missing Value Handling and Feature Scaling

Following the data quality assessment performed during EDA, duplicate observations were retained because the original paper does not describe duplicate removal and it is unclear whether repeated network flows represent legitimate traffic patterns or artifacts of the sampling process.

Before model training, the remaining missing values were replaced using median imputation. As observed during the exploratory data analysis, only 162 missing values were present in the dataset, representing a very small proportion of the data. Consequently, median imputation was sufficient to obtain a complete dataset while minimizing its impact on the overall feature distributions.

Following the methodology proposed in the original paper, Min-Max normalization was then applied to all numerical features, scaling them to the range [0,1]. Feature scaling places all variables on a common scale, preventing features with larger numerical ranges from dominating the learning process and improving the stability of the machine learning algorithms.

6.3.3 Class Balancing Using SMOTE

The exploratory data analysis revealed a severe class imbalance, with some attack categories containing only a small number of samples compared to the BENIGN class. To address this issue, the Synthetic Minority Oversampling Technique (SMOTE) was applied to the training data, following the methodology described in the paper. Applying SMOTE only to the training set prevents information leakage into the test set while allowing the classifiers to learn more representative decision boundaries for the minority classes.

After applying SMOTE, each class contained 18,184 training samples, resulting in a balanced training dataset. This preprocessing step is expected to improve the detection of rare attack types and reduce the tendency of the classifiers to favor the majority classes.

6.3.4 Ensemble Feature Selection

The original paper proposes an ensemble feature selection approach based on four tree-based machine learning models: Decision Tree, Random Forest, Extra Trees, and XGBoost. Feature importance was computed independently for each model, and the four importance scores were averaged to obtain a more reliable estimate of each feature's contribution. The features were then ranked according to their average importance, and features were retained until the cumulative importance reached 90%, exactly following the methodology described by the authors. Although the paper specifies a cumulative feature importance threshold of 90%, it does not justify why this particular threshold was chosen over

alternative values, nor does it experimentally evaluate whether other thresholds could provide a better trade-off between predictive performance and computational efficiency.

In our reproduction, this procedure retained 44 of the 69 non-constant features. Although the original paper reported selecting 36 features, the feature selection algorithm itself was reproduced faithfully. The difference in the number of selected features is likely due to the learned feature importance values or other implementation details that were not fully specified by the authors.

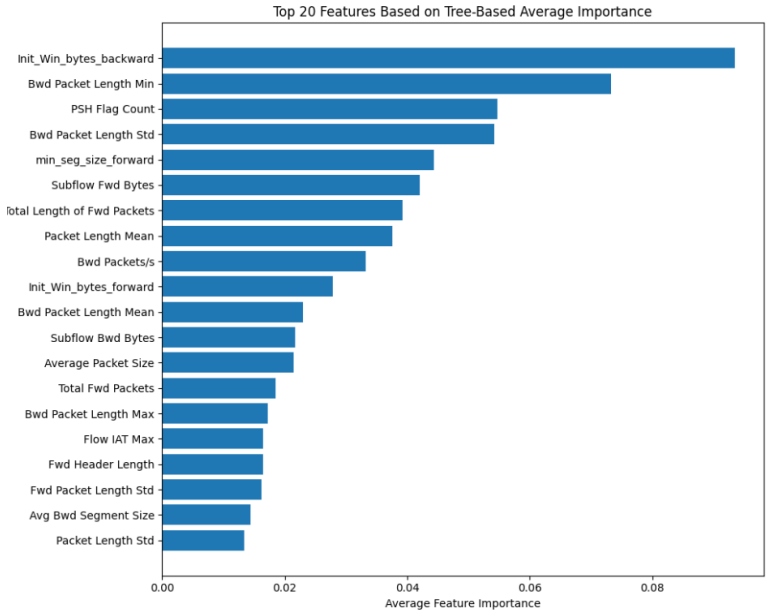

Figure 3: Top 20 feature importance scores.

The selected features primarily describe packet lengths, packet counts, flow durations, timing statistics, TCP flags, and window sizes, all of which provide meaningful information for distinguishing between benign and malicious network traffic.

6.3.5 Feature Creation and Dimensionality Reduction

No additional features were created during this reproduction because the CICIDS2017 dataset already contains a comprehensive set of engineered network-flow characteristics describing packet statistics, timing information, TCP flags, and communication behavior. Similarly, dimensionality reduction techniques such as Principal Component Analysis (PCA) were not applied. Instead, the study follows the methodology of the original paper by reducing the feature space through feature selection rather than feature extraction. This approach preserves the interpretability of the original cybersecurity features while reducing computational complexity.

6.4 Model Training

Following the feature engineering stage, five classification models were trained to reproduce the methodology proposed in the original paper: Decision Tree, Random Forest, Extra Trees, XGBoost, and a Stacking ensemble. The models were trained using the balanced training set produced after applying SMOTE and the 44 selected features obtained through the ensemble feature selection process. To ensure reproducibility, a fixed random seed was used throughout the training process.

The four tree-based classifiers served two purposes in the proposed framework. First, they acted as individual classifiers whose performance could be evaluated independently. Second, following the methodology described in the paper, they formed the first layer of the stacking ensemble. The best-performing base classifier, XGBoost, was automatically selected as the meta-classifier in the second layer of the stacking model. This reproduces the ensemble learning strategy proposed by the authors, where multiple tree-based learners are combined to improve the robustness of the final classifier.

For each model, the training time and prediction time were recorded in addition to the standard classification metrics. Recording both execution times allows a more comprehensive comparison between models, as some classifiers may achieve higher predictive performance at the expense of increased computational cost. The detailed quantitative comparison of the trained models, together with their performance metrics and confusion matrix, is presented in the following Performance Evaluation section.

6.5 Performance Evaluation

6.5.1 Evaluation Methodology

The original paper evaluates the proposed intrusion detection system using Accuracy, Precision, Recall, F1-score, and Execution Time. To ensure a faithful reproduction, the same evaluation metrics were adopted in this study. In addition, the Matthews Correlation Coefficient (MCC) was included because it provides a more informative assessment of classification performance on imbalanced datasets by considering all elements of the confusion matrix. Furthermore, the execution time was separated into training time and prediction time to provide a more detailed analysis of the computational cost of each classifier.

Accuracy measures the proportion of correctly classified network flows and is defined as $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$, where (TP), (TN), (FP), and (FN) denote the numbers of true positives, true negatives, false positives, and false negatives, respectively. Although accuracy provides an overall measure of correctness, it can be misleading for imbalanced intrusion detection datasets because a classifier may achieve high accuracy by correctly classifying the majority class while failing to detect rare attacks.

Precision measures the proportion of predicted attacks that are actually malicious: $Precision = \frac{TP}{TP+FP}$. In the context of intrusion detection, a low precision indicates a high number of false alarms, increasing the workload of security analysts and reducing the practical usability of the system.

Recall measures the proportion of actual attacks that are successfully detected: $Recall = \frac{TP}{TP+FN}$. Recall is particularly important in cybersecurity because false negatives correspond to malicious network traffic that remains undetected. Missing an attack may have serious consequences, making high recall essential for intrusion detection systems.

The F1-score is the harmonic mean of precision and recall: $F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$. Since the CICIDS2017 dataset is highly imbalanced, the macro-averaged F1-score was selected as one of the primary evaluation metrics. Macro averaging assigns equal importance to every attack category regardless of its frequency, providing a more balanced assessment than accuracy alone. This prevents the performance on common classes from masking poor detection of minority attack types such as Infiltration.

Matthews Correlation Coefficient (MCC) is defined as

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

For multi-class classification, MCC is generalized using the confusion matrix and produces a value between -1 and 1, where 1 indicates perfect classification, 0 corresponds to random prediction, and -1 indicates complete disagreement between the predicted and true labels. MCC is particularly suitable for imbalanced datasets because it considers all elements of the confusion matrix rather than focusing only on correctly classified samples. Unlike accuracy, it remains informative even when one or more classes are underrepresented, making it especially appropriate for evaluating intrusion detection systems such as CICIDS2017.

In addition to predictive performance, computational efficiency was evaluated using training time, prediction time, and total execution time. These metrics measure the practical cost of deploying each classifier. While training is performed offline, prediction is executed continuously in real-world intrusion detection systems; therefore, fast prediction time is desirable for real-time network monitoring.

Although additional evaluation metrics such as ROC-AUC and alternative Fβ scores may be useful in some classification problems, they were not included in this study. ROC-AUC is primarily intended for probability-based binary classification and is less interpretable for the multi-class intrusion detection problem considered here. Furthermore, the original paper does not report ROC-AUC, making direct comparison difficult. Similarly, the standard F1-score ($\beta = 1$) was selected instead of alternative Fβ

scores because equal importance was assigned to precision and recall throughout this study, consistent with the evaluation methodology adopted by the authors.

6.5.2 Experimental Results and Comparison with the Original Paper

Table 2: Performance comparison of the trained machine learning models.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	MCC (%)	Training Time (s)	Prediction Time (s)	Total Time (s)
Stacking	99.68	99.55	97.62	98.50	99.55	304.42	0.5073	304.93
XGBoost	99.65	97.34	97.60	97.47	99.50	20.91	0.1680	21.08
Random Forest	95.81	92.11	95.49	93.18	94.21	30.53	0.1007	30.63
Decision Tree	94.13	86.04	94.39	89.54	91.81	2.13	0.0029	2.13
Extra Trees	85.12	71.32	90.90	75.75	81.14	4.53	0.0968	4.63

Table 2 presents the performance of the five reproduced classifiers. Among the individual models, XGBoost achieved the highest predictive performance with an accuracy of 99.65%, a macro F1-score of 97.47%, and an MCC of 99.50%. Following the methodology proposed in the original paper, XGBoost was therefore selected as the meta-classifier in the second layer of the stacking ensemble.

The stacking classifier achieved the best overall performance, obtaining an accuracy of 99.68%, a macro F1-score of 98.50%, and an MCC of 99.55%. Although the improvement in accuracy over XGBoost is relatively small, the increase in both the macro F1-score and the Matthews Correlation Coefficient indicates better overall performance across all traffic classes, including the minority attack categories. Since MCC considers all elements of the confusion matrix and is particularly robust to class imbalance, its improvement further confirms that the stacking ensemble provides a more balanced and reliable classifier than the individual tree-based models. These findings support the authors' claim that combining multiple tree-based classifiers through stacking improves the robustness of the intrusion detection system.

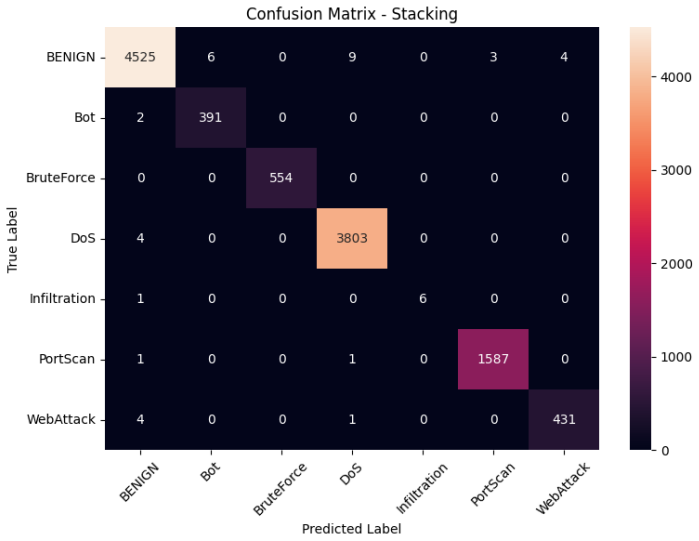

Figure 4: Confusion matrix of the stacking ensemble on the CICIDS2017 test set.

Figure 4 presents the confusion matrix of the stacking model. Most network flows were classified correctly, with only a small number of misclassifications observed. The minority Infiltration class achieved a recall of 0.86 despite containing only seven test samples, illustrating the difficulty of detecting extremely rare attacks. Because each error has a large effect on the evaluation metrics for such a small class, this result remains satisfactory.

Table 3: Comparison between the reproduced results and the results reported in the original paper.

Model	Paper Acc. (%)	Our Acc. (%)	Paper F1 (%)	Our F1 (%)	Paper Time (s)	Our Time (s)
Decision Tree	99.72	94.13	99.80	89.54	126.7	2.42
Random Forest	98.37	95.81	98.30	93.18	2421.6	29.28
Extra Trees	93.43	85.12	93.40	75.75	2349.6	4.81
XGBoost	99.78	99.65	99.70	97.47	1637.2	21.50
Stacking	99.86	99.68	99.80	98.50	4519.3	333.48

Table 3 compares the reproduced results with those reported in the original paper. The reproduced models achieved slightly lower accuracy and macro F1-score than the published results, while following the same preprocessing pipeline and model architecture. These differences are expected because the reproduction was conducted on the sampled CICIDS2017 dataset provided with the authors' implementation rather than the complete dataset, and slight implementation differences or software versions may also influence the learned models.

The computational times reported in this reproduction are substantially lower than those presented in the original paper. This difference is primarily attributed to the use of the sampled dataset, which contains significantly fewer observations than the full CICIDS2017 dataset used by the authors. Consequently, although the absolute execution times are not directly comparable, the relative ranking of the classifiers remains consistent. As in the original study, the stacking ensemble achieved the highest predictive performance while requiring the greatest computational cost.

Although the reproduced performance metrics were slightly lower than those reported in the original paper, direct numerical comparison should be interpreted with caution. The original study reports results obtained using the complete CICIDS2017 dataset, whereas this reproduction evaluates the methodology using the sampled dataset released with the authors' implementation. Therefore, the comparison primarily validates whether the relative performance trends of the proposed framework can be reproduced rather than whether identical numerical results can be achieved.

Our reproduction results support the main conclusions of the original paper. Similar to the authors' findings, the stacking ensemble achieved the highest overall performance among the evaluated classifiers, while XGBoost was the strongest individual model.

6.6 Duplicate Removal Validation

6.6.1 Duplicate Removal Methodology

During the exploratory data analysis, 12,598 duplicated rows (22.2% of the sampled dataset) were identified. Since the original experiments were based on a random stratified train-test split, identical network flows could potentially appear in both the training and testing sets, artificially increasing the reported performance. The original paper does not discuss whether duplicate observations were removed prior to model training. To evaluate the impact of this potential source of bias, an additional validation experiment was conducted.

All duplicated rows were removed from the dataset before any preprocessing steps. The entire experimental pipeline was then repeated without modification, including train-test splitting, missing value imputation, Min-Max normalization, SMOTE applied only to the training set, ensemble feature selection using the 90% cumulative importance threshold, and the training and evaluation of all five classifiers. This ensured that the only intentional change was the removal of duplicated observations. All preprocessing, feature engineering, feature selection, and model-training procedures were repeated using the duplicate-removed dataset.

6.6.2 Duplicate Removal Evaluation

Table 4: Comparison of the Stacking classifier before and after duplicate removal.

Experiment	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	MCC	Training Time (s)	Prediction Time (s)	Total Time (s)
Original sample with duplicates	99.68	99.55	97.62	98.50	0.9955	474.06	0.9146	474.97
Duplicate-removed sample	99.53	96.37	99.10	97.67	0.9924	522.21	0.7769	522.99

Table 4 compares the performance of the best-performing classifier (Stacking) before and after duplicate removal. Removing duplicated observations reduced the dataset size from 56,661 to 44,063 samples. The ensemble feature selection process selected 43 features instead of 44, indicating that duplicate removal had little influence on the identified important features.

The duplicate-removal experiment produced only a small decrease in performance. The accuracy decreased from 99.68% to 99.53%, the macro F1-score decreased from 98.50% to 97.67%, and the MCC decreased from 0.9955 to 0.9924. Although these results indicate that duplicated observations contributed to slightly higher evaluation scores, the overall performance of the proposed framework remained extremely high after duplicate removal. Therefore, the additional experiment suggests that duplicated observations caused a measurable but relatively small inflation of the reported evaluation metrics. Nevertheless, the framework remained highly effective under this stricter evaluation protocol, indicating that the strong performance reported in this study is not solely attributable to duplicated observations.

7 Error Analysis

Figure 4 presents the confusion matrix of the stacking classifier. It indicates that the vast majority of network flows were classified correctly, with only a small number of misclassifications across all traffic classes. Most errors occurred between attack categories with similar network-flow characteristics, while the BENIGN, DoS, and PortScan classes achieved almost perfect classification. The most challenging class was "Infiltration", which contained only seven test samples. One of these samples was misclassified, resulting in a recall of 0.86. Because of the extremely limited number of observations, even a single classification error has a noticeable impact on the evaluation metrics for this class.

The observed error pattern is closely related to the characteristics of the dataset. The severe class imbalance present in the original CICIDS2017 dataset means that minority attack classes provide far fewer training examples than the majority classes. Although SMOTE substantially improved class balance during training, the test set still contained only a small number of Infiltration samples. Consequently, the classifier had less information available to learn the unique characteristics of these rare attacks, making them more difficult to distinguish from normal traffic or other attack categories.

From a cybersecurity perspective, different types of classification errors have different practical consequences. A "False Positive" occurs when legitimate network traffic is incorrectly classified as malicious. Although this increases the number of security alerts that require investigation, it does not directly compromise the security of the monitored system. In contrast, a "False Negative" occurs when malicious traffic is incorrectly classified as benign, allowing an attack to remain undetected. For intrusion detection systems, False Negatives are generally considered more critical because they may enable attackers to compromise the system without triggering an alarm.

The duplicate-removal validation experiment further supports these observations. Although removing 12,598 duplicated flows (22.2% of the sampled dataset) resulted in a slight decrease in overall Accuracy, F1-score, and MCC, the overall error pattern remained largely unchanged. This suggests that the remaining classification errors are primarily associated with the intrinsic difficulty of distinguishing minority attack classes rather than being caused by duplicated observations in the dataset.

Both the original reproduction and the duplicate-removal validation experiment demonstrate a favorable trade-off between False Positives and False Negatives. The stacking ensemble achieved very high precision while maintaining high recall across all attack categories, indicating that it successfully detected most malicious traffic without generating an excessive number of false alarms. Although duplicate removal produced a small reduction in the overall evaluation metrics, the error characteristics remained similar, suggesting that the proposed framework is robust to the removal of duplicated observations. Nevertheless, the remaining errors observed for the minority classes indicate that detecting rare attacks remains one of the primary challenges in network intrusion detection. Future work should therefore focus on improving the recognition of low-frequency attack types through additional data collection, more representative datasets, alternative sampling techniques, or more diverse ensemble learning strategies.

8 Conclusions

This study reproduced the intrusion detection framework proposed in the selected paper using the CICIDS2017 sample dataset provided by the authors. The complete machine learning pipeline, including data preprocessing, exploratory data analysis, feature engineering, model training, error analysis, and performance evaluation, was successfully reproduced. In addition, an extra validation experiment was conducted after removing duplicated observations to assess the impact of duplicate flows on the validity of the reported performance. Overall, the reproduced results closely followed the findings reported in the original study.

Since this study was conducted using the sampled CICIDS2017 dataset rather than the complete dataset used in the original paper, the reproduced results should be interpreted as validating the proposed methodology on the sampled data rather than directly reproducing the published numerical results.

The experimental results demonstrated that the stacking ensemble achieved the highest overall performance among the evaluated classifiers, while XGBoost was the strongest individual model. Although the reproduced performance metrics were slightly lower than those reported in the paper, the ranking of the classifiers remained consistent, supporting the authors' main claim that combining multiple tree-based classifiers improves intrusion detection performance. The reproduction also confirmed that the ensemble feature selection strategy effectively reduced the dimensionality of the dataset while preserving predictive performance.

Several lessons were learned throughout the reproduction process. First, careful data preprocessing, including handling missing values, balancing the training data using SMOTE, and selecting informative features, plays an essential role in achieving high classification performance. Second, the study highlighted the importance of reproducibility, as several implementation details described in the paper were either incomplete or not fully reflected in the released source code, requiring additional investigation during reproduction. Finally, the duplicate-removal validation experiment demonstrated the importance of evaluating potential sources of bias in benchmark datasets. Although removing duplicated observations caused a slight decrease in the evaluation metrics, the overall conclusions remained unchanged, indicating that the proposed framework is robust while emphasizing the need for careful validation of experimental results.

The proposed framework has several strengths. It combines high predictive performance with an interpretable tree-based machine learning pipeline and employs an effective ensemble feature selection strategy that reduces computational complexity. However, the critical evaluation also identified several limitations. The stacking ensemble relies exclusively on tree-based learners, limiting model diversity, the rationale for selecting a 90% cumulative feature importance threshold is not experimentally justified, and the potential influence of duplicated or redundant observations is not discussed despite their presence in the dataset. Furthermore, the evaluation is limited to benchmark datasets and does not investigate temporal generalization or concept drift, which are important considerations for real-world intrusion detection systems. The additional validation experiment also increased confidence that the reported performance trends are genuine and not primarily an artifact of duplicated observations.

Future work may investigate alternative feature selection thresholds, compare more diverse ensemble architectures, evaluate different feature importance strategies, and validate the framework on additional and more recent intrusion detection datasets. Such experiments would provide a more comprehensive understanding of the robustness, generalizability, and practical applicability of the proposed methodology.

9 Summary

This project reproduced and critically evaluated the paper *Tree-based Intelligent Intrusion Detection System in Internet of Vehicles*. The paper addresses the problem of network intrusion detection by combining ensemble feature selection with a stacking-based machine learning framework to accurately distinguish between benign and malicious network traffic.

The reproduction was conducted using the CICIDS2017 sample dataset provided with the authors' implementation. The complete methodology described in the paper was reproduced, including exploratory data analysis, data preprocessing, feature engineering, ensemble feature selection, model training, error analysis, and performance evaluation. To strengthen the evaluation, an additional vali-

dation experiment was performed after removing duplicated observations from the sampled dataset to assess their impact on the reported performance. Five classifiers were trained: Decision Tree, Random Forest, Extra Trees, XGBoost, and a stacking ensemble classifier. Their performance was evaluated using Accuracy, Precision, Recall, F1-score, Matthews Correlation Coefficient (MCC), and execution time.

The reproduced results followed the main performance trends reported in the original paper. Similar to the authors' work, XGBoost achieved the best performance among the individual classifiers, while the stacking ensemble obtained the highest overall predictive performance. Although the reproduced performance metrics were slightly lower than those reported in the paper, the overall ranking of the classifiers remained consistent, supporting the effectiveness of the proposed ensemble learning approach. The duplicate-removal validation experiment further demonstrated that the strong performance of the framework was not solely attributable to duplicated observations, as only a small decrease in the evaluation metrics was observed after removing 22.2% duplicated flows from the sampled dataset.

The critical evaluation concluded that the authors' main claims are generally supported by both the evidence presented in the paper and the results obtained in this reproduction. At the same time, several methodological limitations were identified. These include the exclusive use of tree-based base learners in the stacking ensemble, the lack of experimental justification for the selected 90% feature importance threshold, limited discussion of dataset quality and redundancy, and the absence of validation on independent or temporally evolving datasets.

One of the most important insights obtained during this study is that careful data preprocessing and feature selection contribute significantly to the success of intrusion detection models. The reproduction also demonstrated that reproducibility extends beyond executing the released code, requiring careful verification of preprocessing decisions and methodological assumptions that were not fully documented in the original paper. In particular, the duplicate-removal validation experiment highlighted the importance of evaluating potential threats to experimental validity. Although duplicated observations produced a modest improvement in the reported performance, the overall conclusions remained unchanged, indicating that the proposed framework is robust while emphasizing the need for transparent dataset preprocessing.

Overall, the proposed framework is recommended for similar supervised intrusion detection problems because it combines strong predictive performance with a relatively interpretable machine learning pipeline. However, future studies should validate the approach on additional cybersecurity datasets, investigate alternative feature selection thresholds, and explore more diverse ensemble architectures to further improve robustness and generalizability.

In conclusion, this reproduction successfully confirmed the main findings of the original paper while providing a more comprehensive assessment of its methodology, assumptions, and limitations. The study demonstrates that the proposed stacking ensemble framework is an effective solution for network intrusion detection and provides a strong baseline for future cybersecurity machine learning research.